

MODULE 13 · REST APIS · BACKEND DEVELOPMENT

Authentication & API Security

How do we make sure the right users access the right APIs?



Security is a journey, not a single lock

A request becomes trustworthy step by step—before the API ever returns data.

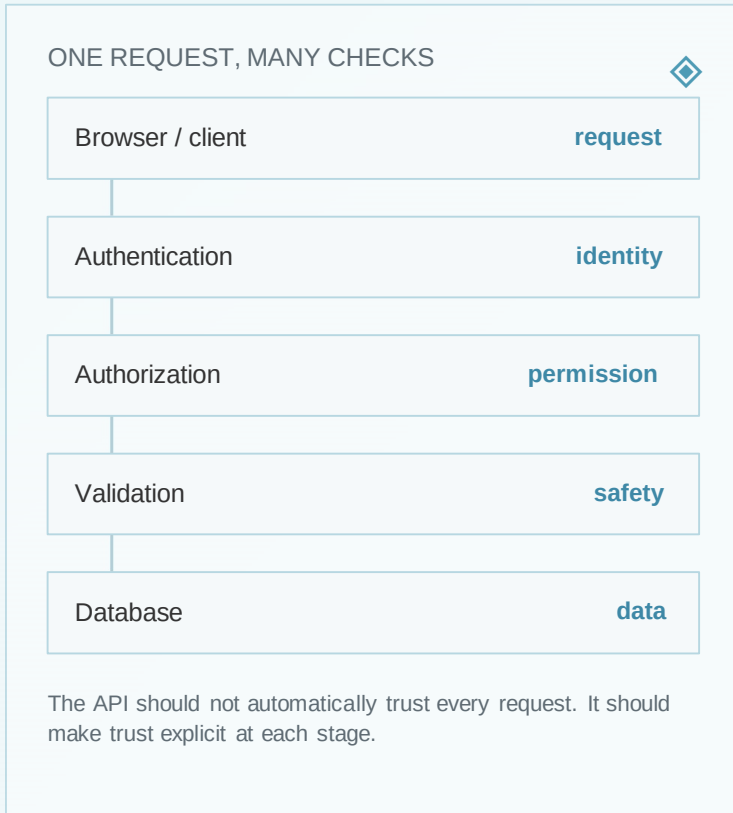
Who are you?

What may you do?

Is this request safe?

- 1 User / browser → HTTPS
- 2 Authentication → Token / JWT
- 3 API request → Authorization
- 4 Validation → Resource

Ask students: What could go wrong if `/users` or `/payments` accepted every request?

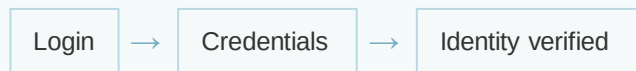


Authentication proves identity; authorization limits access

WHO? → WHAT?

WHO are you?

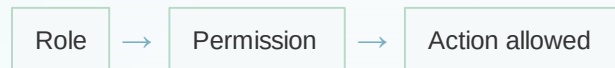
Analogy: Showing your ID at the building entrance.



Technical check: “Is this really Student A?”

WHAT may you do?

Analogy: The guard decides which rooms your ID can open.



Technical check: “Can Student A access these grades?”

University example: Student logs in → own grades allowed → another student’s grades denied.

Ask students: Does a valid login mean full access?

A token is a temporary digital access pass

Token lifecycle: Login → Server verifies credentials → Server issues token → Client stores token → Future requests present token.

This means the client does not need to resend the password on every single request.



JSON Web Token (JWT) is a compact token format that carries claims about the user.



It can be **integrity-protected** with a signature to detect if it has been tampered with.



Warning: It is NOT automatically encrypted. Never place passwords or API secrets in the payload.

HEADER

How the token is handled (e.g., algorithm)

PAYLOAD

Claims and data (e.g., sub, role, exp)

SIGNATURE

Helps verify the token has not been altered

The request flow: login once, prove it every time

A token acts as a temporary digital access pass. It allows the client to authenticate subsequent requests without resending the user's password every time.



1. Initial Login

User enters credentials; client sends a POST request to the login endpoint.



2. Token Issuance

Server verifies the credentials, generates a signed token, and returns it to the client.



3. Authenticated Request

Client includes the token in the the Authorization header of all future API requests.



4. Validation & Access

API validates the token's signature and expiration before returning the requested data.

GET /api/profile HTTP/1.1

Host: api.university.edu

Accept: application/json

Authorization: Bearer eyJhbGciOi...

?????

What should happen when the token is missing, expired, or altered?
altered?

Permissions turn identity into safe actions

Decision pipeline: Authenticated user → Identify role → Check action → action → Check resource ownership → Allow or Deny.



Student Role

View own profile and grades. Cannot create or delete records.



Teacher Role

View and update student records. Cannot delete users.



Admin Role

Full access to manage users, roles, and system settings.

Failure Example:

```
DELETE /api/users/25
```

Valid student token + insufficient permission = **403 Forbidden**



Security is layered: protect transport, inputs, and capacity

API security is a collection of practices, not a single feature. Multiple complementary layers protect the system from different types of attacks.



HTTPS (Transport)

Encrypts communication between the client and server, protecting data while it travels.



Input Validation

Never blindly trusts request data. Validates, sanitizes, or rejects invalid input before processing.



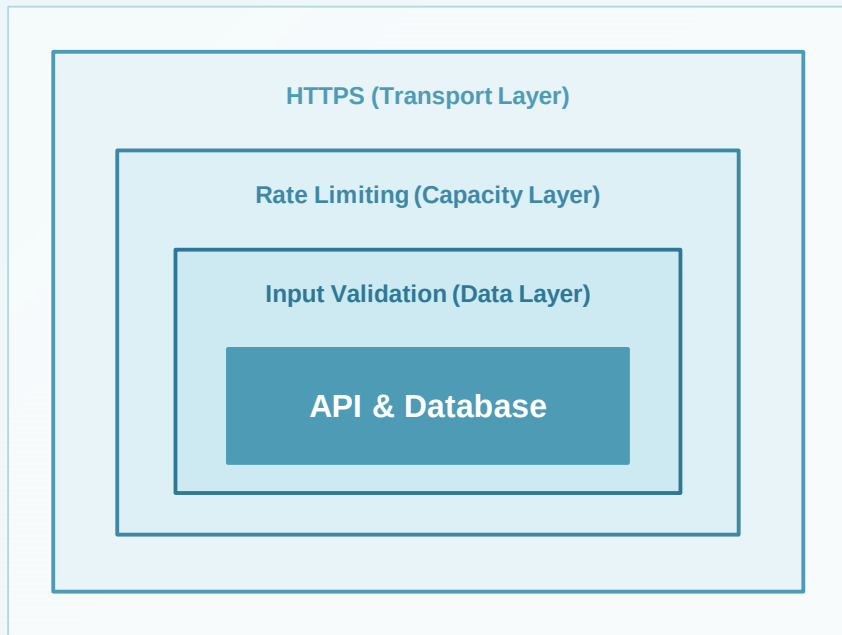
Rate Limiting

Controls how many requests a client can make, preventing abuse and resource exhaustion.



Secrets Management

Keeps API keys and signing secrets in environment variables, never in frontend code.



CORS is the browser's cross-origin gate

CORS controls which browser origins can make or read cross-origin requests. It's enforced by the browser, not the API.



Understanding Origin

Origin = scheme + host + port (e.g. <http://localhost:3000>).



Browser Enforcement

The browser checks the API's CORS policy before allowing frontend code to read responses.



The Response Header

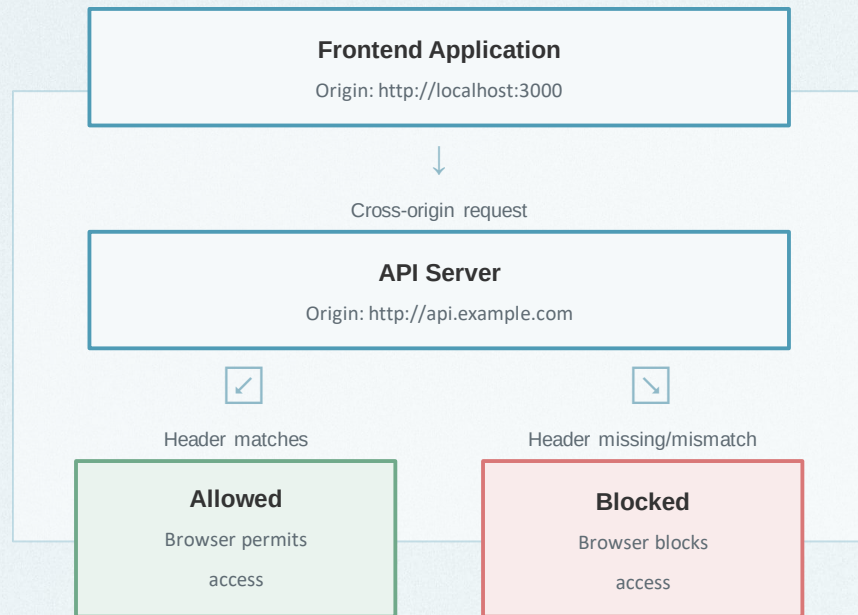
APIs grant access via headers like [Access-Control-Allow-Origin](#).



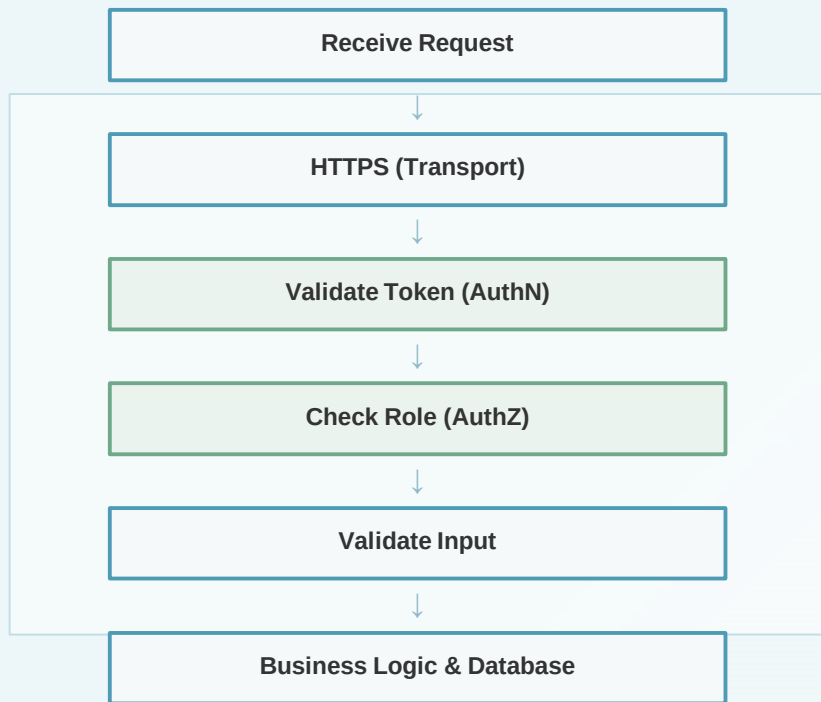
Not Authentication

CORS doesn't verify identity and doesn't protect non-browser clients (e.g. curl).

Ask students: Which layer answers “Which browser origins are allowed?”



Secure the Student Management API



Apply the security concepts to a practical API pipeline. Different roles require different levels of access to the endpoints.



Student Role

Can view own profile and grades.

`GET /students/:id`



Teacher Role

Can view and update student records.

`POST /students`



Admin Role

Can manage users and roles.

`DELETE /students/:id`

SCENARIO CHALLENGE

A logged-in student requests `DELETE /students/25`.

Authenticated = Yes. Authorized = No.

Response = **403 Forbidden**.

Remember the flow; then test your design

Takeaway: Secure APIs combine identity, permissions, transport protection, validation, rate limiting, and careful secret handling.

FINAL REVIEW

- ? What is the difference between authentication and authorization?
- ? What are the three parts of a JWT?
- ? Why is a JWT payload not automatically secret?
- ? When should an API return 401 versus 403?
- ? What exactly does CORS control?
- ? Why must permissions be checked after authentication?

